
Security Vulnerability Report

Synthra V3 — Tolly Labs

Finding Summary

ID	Severity	Protocol	Description
SYN-01	Critical	Synthra V3	Double protocol fee accounting → pool insolvency
SYN-02	Critical	Synthra V3	Pre-callback fee transfer DoS
SYN-03	High	Synthra V3	Flash loan double fee accounting
SYN-04	High	Synthra V3	FeeProtocol configuration lockout
SYN-05	High	Synthra V3	Immutable zero-address fee recipient
TOL-01	Critical	Tolly Labs	Burn target blocked by max-wallet cap
TOL-03	Critical	Tolly Labs	Interface mismatch in fee collection

Report Date: 19 September 2026

Total Findings: 7 (4 Critical, 3 High)

Disclosure: Responsible disclosure — no active bug bounty for either project

1. Synthra V3 Findings

SYN-01: Double Protocol Fee Accounting (Critical)

Files: `SynthraV3Pool.sol:765-767, 867-886`

The custom protocol-fee mechanism performs two conflicting operations on every swap. First, it records the fee amount in the `protocolFees` storage mapping. Then it immediately transfers the same amount to the fee recipient. When the factory owner later calls `collectProtocol()`, the function reads the same storage value and transfers the tokens a second time. The pool is debited twice for the same fee.

Impact: **Pool insolvency.** Liquidity providers cannot withdraw their positions because the pool balance has been depleted by the extra transfer. The pool balance falls below the theoretical reserve.

Fix: Remove the storage accrual and rely solely on the immediate transfer, or remove the immediate transfer and let `collectProtocol()` handle all fee distribution. Both are mutually exclusive in the Uniswap V3 design.

SYN-02: Pre-Callback Fee Transfer Causes DoS (Critical)

Files: `SynthraV3Pool.sol:767, 787`

The `safeTransfer` for protocol fees executes before the swap callback. The callback is where the swapper transfers input tokens to the pool. If the pool has no preexisting balance for the input token — as is the case for first swaps, single-sided pools, or after a full withdrawal — the transfer reverts before the callback even runs.

Impact: **Swap denial of service.** Legitimate swaps fail with a transfer error even though the swapper would have supplied the full amount moments later.

Fix: Move the protocol fee transfer after the swap callback, or deduct the fee directly from the callback-supplied funds.

SYN-03: Flash Loan Double Fee Accounting (High)

Files: `SynthraV3Pool.sol:835-837`

The same double-accounting pattern from SYN-01 is present in the `flash()` function. Protocol fees on flash loans are both recorded in storage and immediately transferred, enabling the factory owner to collect them a second time.

Impact: The factory owner can drain protocol fees beyond what was collected on flash loans.

Fix: Same as SYN-01 — storage accrual and immediate transfer cannot both exist.

SYN-04: FeeProtocol Configuration Lockout (High)

Files: SynthraV3Pool.sol:287, 856-860

The `initialize()` function sets `feeProtocol = 3 + (3 << 4)`, encoding a 1/3 protocol fee share. However, `setFeeProtocol()` rejects any value below 4. If governance ever disables the fee (sets to 0), the original 1/3 configuration can never be restored.

Impact: Irreversible protocol fee configuration. Governance loses flexibility.

Fix: Accept value 3 in the setter constraint, or initialize with an acceptable value (4 or higher).

SYN-05: Immutable Zero-Address Fee Recipient (High)

Files: SynthraV3Pool.sol:57, SynthraV3Factory.sol:67-71

The fee recipient is declared `immutable` in each pool. The factory's `setFeeRecipient()` has its zero-address check commented out. A pool deployed with `address(0)` as the fee recipient cannot be repaired, and all fee-bearing swaps revert.

Impact: Permanently broken pools. No on-chain fix is possible.

Fix: Uncomment the zero-address check. Consider making the fee recipient mutable via proxy.

2. Tolly Labs Findings

TOL-01: Burn Target Blocked by Max-Wallet Cap (Critical)

Files: `TollyToken.sol:61-64, 78-86`

The anti-snipe mechanism exempts the pad, position manager, locker, and pool from the max-wallet cap. The canonical burn address (`0x0000...dEaD`) is not exempt. Once the DEAD address holds more than `maxWallet` (30 million tokens, 3% of supply), any transfer to it — including fee burning, buyback-and-burn, or swap routings — reverts with `MaxWalletExceeded()`.

Impact: **Core mechanism DoS.** The buyback-and-burn and any fee-redistribution functions targeting the burn address are blocked for the duration of the anti-snipe window.

Fix: Add DEAD to `antiSnipeExempt`, or exempt recognized burn destinations from the wallet-limit check.

TOL-03: Fee Collection Interface Mismatch (Critical)

Files: `TollyForge.sol:98-100, 850, 1863, 2331; TollyFeeLocker.sol:225`

The `ILaunchLocker` interface expects `collectFees(address)`. The fee locker implements `collect(uint256)`. These have different EVM function selectors. Every call from the Forge contracts wraps in a `try/catch` block, which silently catches the revert and logs zero fees collected.

Impact: **Zero fee harvesting.** LP fee revenue is never collected through the Forge contracts. The `try/catch` makes the failure invisible to on-chain monitoring.

Fix: Standardize the function name and parameter type across the interface, implementation, and all call sites.

3. Verification Methodology

Each finding was verified through the following process:

1. **Source code review** — Direct reading of the affected contract files with line-level reference.
2. **Independent AI validation** — A separate LLM auditor reviewed the same source code and confirmed each finding.
3. **Cross-validation** — Findings were independently validated through a secondary model (GPT-5.6 Luna) to eliminate hallucination risk.

4. Disclaimer

This report reflects the findings of a security review conducted on the public source code repositories listed above. No testing was performed against production deployments. The severity ratings reflect the potential economic impact under reasonable attack scenarios. The authors are not affiliated with either project.